Java Multithreading and File Handling

Student-friendly revision guide • 30 questions • Viva-ready answers

A. Multithreading in Java

1. What is a thread in Java?

Answer: A lightweight unit of execution within a process. Multiple threads can run independently while sharing the process's memory.

2. What is multithreading?

Answer: Running two or more threads concurrently in one program, allowing tasks to make progress at the same time.

3. Why is multithreading used in Java applications?

Answer: It improves responsiveness and throughput, such as keeping a UI responsive while work runs in the background or serving many client requests.

4. What is the difference between a process and a thread?

Answer: A process has its own memory and resources. A thread is a smaller execution path inside a process and shares that process's memory.

5. How can you create a thread in Java?

Answer: Extend `Thread` and override `run()`, or implement `Runnable` and pass it to a Thread. Modern applications often use an ExecutorService.

6. What is the difference between extending Thread and implementing Runnable?

Answer: Extending Thread prevents extending another class. Runnable separates the task from the thread and is more flexible, so it is usually preferred.

7. What is the purpose of the start() method?

Answer: `start()` asks the JVM to create a new thread of execution, which then calls that thread's `run()` method.

8. What is the difference between start() and run()?

Answer: `start()` begins a new thread. Calling `run()` directly is an ordinary method call that runs on the current thread.

9. What is the life cycle of a thread?

Answer: A thread moves through NEW, RUNNABLE, BLOCKED/WAITING/TIMED_WAITING, and TERMINATED states.

10. What is the purpose of sleep()?

Answer: `Thread.sleep(time)` pauses the current thread for at least the specified time. It does not release any monitor lock it holds.

11. What is the difference between sleep() and wait()?

Answer: `sleep()` pauses for time and keeps locks. `wait()` is called while holding an object's monitor, releases that monitor, and waits for notification or timeout.

12. What is thread synchronization?

Answer: Coordinating access to shared data so only safe operations occur at a time and results remain consistent.

13. Why is the synchronized keyword used?

Answer: It provides mutual exclusion for a block or method and establishes visibility guarantees between threads using the same monitor.

14. What is a race condition?

Answer: A bug where the outcome depends on unpredictable timing between threads, often because they read and modify shared data without proper coordination.

15. What is the difference between join() and sleep()?

Answer: `join()` makes one thread wait for another to finish. `sleep()` pauses the current thread for a duration regardless of other threads.

B. File Handling in Java

16. What is file handling in Java?

Answer: Creating, reading, writing, deleting, and managing files and directories from a Java program.

17. Which package is commonly used for file handling in Java?

Answer: `java.io` provides classic file and stream classes. `java.nio.file` provides the modern Path and Files APIs.

18. What is the purpose of the File class?

Answer: `java.io.File` represents a file or directory path and lets you inspect metadata or perform operations such as create, delete, and list.

19. How do you create a new file using Java?

Answer: Use `new File("notes.txt").createNewFile()`; handle `IOException`. With NIO, use `Files.createFile(path)`.

20. How do you check whether a file exists?

Answer: Use `file.exists()` for a File, or `Files.exists(path)` for a Path.

21. How do you obtain the name and path of a file?

Answer: Use `file.getName()`, `file.getPath()`, or `file.getAbsolutePath()`. For Path, use methods such as `getFileName()`.

22. What is the difference between FileReader and FileWriter?

Answer: FileReader reads characters from a text file; FileWriter writes characters to a text file. Both use the platform default charset unless a charset is specified through newer APIs.

23. What is the difference between character streams and byte streams?

Answer: Character streams (`Reader`/`Writer`) handle text and character encoding. Byte streams (`InputStream`/`OutputStream`) handle raw binary data.

24. What is the purpose of FileInputStream?

Answer: It reads raw bytes from a file, making it suitable for binary content such as images or PDFs.

25. What is the purpose of FileOutputStream?

Answer: It writes raw bytes to a file, commonly for binary content. It can append when constructed with the append flag.

26. What is the difference between BufferedReader and FileReader?

Answer: FileReader reads characters directly. BufferedReader wraps a Reader with buffering and convenient methods such as `readLine()`, improving efficiency for repeated reads.

27. How do you read a file line by line in Java?

Answer: Use a BufferedReader in try-with-resources and call `readLine()` until it returns null. `Files.lines(path)` is another option.

28. How do you write data into a text file?

Answer: Use a Writer such as BufferedWriter or FileWriter in try-with-resources, then call `write()`. `Files.writeString(path, text)` is a concise modern alternative.

29. Why should files be closed after reading or writing?

Answer: Closing releases operating-system resources, flushes buffered output, and prevents file-handle leaks. Try-with-resources closes automatically.

30. What exceptions can occur during file handling, and how can they be handled?

Answer: Common exceptions include `IOException`, `FileNotFoundException`, and security-related exceptions. Handle them with try-catch or declare them with `throws`; prefer try-with-resources for cleanup.